

Chapter 7: Supervised Learning - Model Estimation

100 Tutorial Topics: OLS, NLS, MLE, Gradient Descent, Backpropagation, Regularization, Cross-Validation

GARP RAI Exam Preparation

T1: Model Estimation Purpose

TOPIC: Primary Purpose of Model Estimation in Supervised Learning

DEFINITION:

- Model estimation is the essential step where a model learns from data
- It involves finding the specific parameter values that cause the model to best fit the training data
- Parameters are often called "weights" in machine learning

TWO MAIN ESTIMATION FAMILIES:

- 1 **Least Squares:** Minimize sum of squared differences between predictions and actual values
 - Objective: Make prediction errors (residuals) as small as possible
- 2 **Maximum Likelihood:** Probabilistic approach
 - Assume data comes from specific distribution
 - Find parameters that make observing data most likely

WHY IT MATTERS:

- Fundamental to all supervised learning
- Determines model performance
- Foundation for prediction and classification

EXAM TRAP: Model estimation is about finding **optimal parameter values**, not increasing complexity!

T2: Analytical vs Numerical Methods

TOPIC: Distinguishing Between Analytical and Numerical Estimation Methods

DEFINITION:

- Estimation methods can be broadly classified into analytical and numerical approaches
- The choice depends on the complexity of the objective function and model

ANALYTICAL METHODS:

- **Closed-form solution:** Exact mathematical equations
- **Unique solution:** Parameter values are unique
- **Example:** OLS for linear regression: $\hat{\beta} = (X^T X)^{-1} X^T y$
- **When available:** When objective function can be differentiated

NUMERICAL METHODS:

- **No closed-form solution:** Iterative procedures
- **Start with initial guess:** Gradually move toward optimal
- **Example:** Gradient descent, NLS
- **When needed:** When differentiation is too complex or impossible

KEY INSIGHT: Analytical = exact formula; Numerical = iterative approximation

EXAM TRAP: Analytical methods have **closed-form solutions**; numerical methods require **iterative procedures**!

T3: Hyperparameters vs Parameters

TOPIC: Distinguishing Between Hyperparameters and Parameters

DEFINITION:

- Parameters are learned from data during training
- Hyperparameters are configuration settings chosen before training begins

PARAMETERS (WEIGHTS):

- **Learned from data:** Estimated by optimization algorithm
- **Examples:**
 - Regression coefficients ($\beta_0, \beta_1, \dots$)
 - Neural network weights
 - Bias terms
- **Updated during training:** Changed with each iteration

HYPERPARAMETERS:

- **Set before training:** Configuration settings
- **Examples:**
 - Learning rate (η)
 - Number of hidden layers
 - Regularization parameter (λ)
 - Number of epochs
- **Require tuning:** Grid search, cross-validation

EXAM TRAP: Parameters = **learned from data**; Hyperparameters = **set before training**!

TOPIC: Overview of the Least Squares Method

DEFINITION:

- Technique to estimate parameters by minimizing sum of squared differences between observed values and model predictions
- Two variants: Ordinary Least Squares (OLS) and Nonlinear Least Squares (NLS)

OBJECTIVE FUNCTION:

- Residual Sum of Squares (RSS): $\sum_{i=1}^N (y_i - \hat{y}_i)^2$
- Mean Squared Error (MSE): $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$

WHY SQUARE RESIDUALS?

- Positive and negative residuals would cancel if simply summed
- Sum of residuals is always zero at optimal solution
- Squaring ensures all residuals contribute positively
- Penalizes large errors more heavily

KEY CHARACTERISTICS:

- **OLS**: Linear in parameters; closed-form solution
- **NLS**: Nonlinear in parameters; iterative solution

EXAM TRAP: Least Squares minimizes **squared** differences, not absolute differences!

TOPIC: Overview of Maximum Likelihood Estimation

DEFINITION:

- Probabilistic approach to parameter estimation
- Finds parameter values that maximize probability of observing the given data
- More flexible framework than Least Squares

CORE QUESTION:

- "Given observed data, what parameter values make this data most probable?"

PROCESS:

- 1 Assume data comes from specific distribution
- 2 Construct likelihood function: $L(\theta) = P(X|\theta)$
- 3 Find θ that maximizes $L(\theta)$
- 4 Use log-likelihood for easier computation

APPLICATIONS:

- Logistic regression (Bernoulli distribution)
- Linear regression (Normal distribution)
- Probit models
- GARCH models for volatility

EXAM TRAP: MLE maximizes **likelihood** = $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$!

TOPIC: Ordinary Least Squares - Definition and Overview

DEFINITION:

- Most straightforward estimation method
- Analytical approach with closed-form solution
- Used to estimate parameters in linear regression models

OBJECTIVE:

- Minimize Residual Sum of Squares (RSS)
- $RSS = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$

KEY CHARACTERISTICS:

- **Analytical method:** Closed-form solution
- **Linear in parameters:** $y = \beta_0 + \beta_1 x + \varepsilon$
- **Unique solution:** One optimal set of parameters
- **Computationally efficient:** Direct calculation

EXAMPLE:

- Salary = 16.88 + 1.06 × Experience
- Intercept = 16.88, Slope = 1.06

EXAM TRAP: OLS is an **analytical** method with **closed-form solution**!

T7: OLS - Residual Sum of Squares

TOPIC: Residual Sum of Squares (RSS) in OLS

DEFINITION:

- RSS is the sum of squared differences between actual and predicted values
- It is the objective function that OLS minimizes

FORMULA:

$$RSS = \sum_{i=1}^N \hat{u}_i^2 = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

where:

- y_i = actual observed value
- $\hat{y}_i$ = predicted value
- $\hat{u}_i = y_i - \hat{y}_i$ = residual

RELATED METRICS:

- $MSE = RSS/N$ (Mean Squared Error)
- $RMSE = \sqrt{MSE}$ (Root Mean Squared Error)

WHY SQUARE?

- Prevents positive and negative residuals from canceling
- Sum of residuals is always zero at OLS solution
- Penalizes large errors more heavily

EXAM TRAP: RSS = sum of squared residuals, NOT sum of absolute residuals!

TOPIC: Mean Squared Error (MSE) in OLS

DEFINITION:

- MSE is the average of squared residuals
- Commonly used metric for model performance

FORMULA:

$$MSE = \frac{1}{N} \sum_{i=1}^N \hat{u}_i^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{RSS}{N}$$

RELATIONSHIP TO RSS:

- $MSE = RSS / N$
- Both minimized at same parameter values
- MSE is scale-independent of sample size

WHY MSE IS USEFUL:

- Easier to compare across datasets
- Commonly used as loss function in machine learning
- Basis for RMSE calculation

EXAM TRAP: $MSE = RSS/N$ = average squared error!

T9: OLS - Fitted Line Example

TOPIC: Fitted Regression Line in Salary Example

EXAMPLE DATA:

- 14 observations of bank employee experience and salary
- Simple linear regression: one feature (experience), one target (salary)

FITTED EQUATION:

$$\hat{y} = 16.88 + 1.06x$$

where:

- $\hat{y}$ = predicted salary (USD/hour)
- x = years of experience
- Intercept $\hat{\beta}_0 = 16.88$
- Slope $\hat{\beta}_1 = 1.06$

INTERPRETATION:

- Base salary (0 experience): \$16.88/hour
- Each additional year: +\$1.06/hour

EXAMPLE PREDICTION:

- Experience = 9 years
- Predicted salary = $16.88 + 9(1.06) = 26.42$

EXAM TRAP: Know the **intercept and slope** from the example!

TOPIC: Calculating Residuals in OLS

DEFINITION:

- Residual = Actual value - Predicted value
- $\hat{u}_i = y_i - \hat{y}_i$

EXAMPLE:

- Data point: Experience = 9 years, Salary = \$33.08/hour
- Predicted: $\hat{y} = 16.88 + 9(1.06) = 26.42$
- Residual: $33.08 - 26.42 = 6.66$

INTERPRETATION:

- Positive residual: actual $\geq$ predicted
- Model under-predicted for this point
- Negative residual: actual $\leq$ predicted
- Model over-predicted for this point

PROPERTY:

- Sum of residuals = 0 at OLS solution
- $\sum \hat{u}_i = 0$

EXAM TRAP: Residual = actual - predicted, NOT predicted - actual!

TOPIC: Deriving OLS Estimates Using Normal Equations

DERIVATION PROCESS:

- 1 Write RSS as function of parameters
- 2 Partially differentiate with respect to each parameter
- 3 Set derivatives to zero (first-order conditions)
- 4 Solve system of equations (normal equations)

RESULTING FORMULAS:

- Intercept: $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$
- Slope: $\hat{\beta}_1 = \frac{\sum_{i=1}^N y_i x_i - N \bar{y} \bar{x}}{\sum_{i=1}^N x_i^2 - N \bar{x}^2}$

WHY THIS WORKS:

- RSS is convex function of parameters
- Unique minimum exists
- First-order conditions characterize minimum

KEY INSIGHT:

- OLS has closed-form solution
- No iterative optimization needed
- Direct calculation from data

EXAM TRAP: OLS uses **calculus** (derivatives) to find closed-form solution!

TOPIC: Extending OLS to Multiple Regression

DEFINITION:

- Multiple regression has more than one explanatory variable
- Model: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m + \varepsilon$

MATRIX FORMULATION:

- Model: $y = X\beta + \varepsilon$
- Normal equations: $X^T X \hat{\beta} = X^T y$
- Solution: $\hat{\beta} = (X^T X)^{-1} X^T y$

WHY MATRIX ALGEBRA:

- Formulas become complex with many variables
- Matrix notation is compact
- Computationally efficient

REQUIREMENTS:

- $X^T X$ must be invertible
- No perfect multicollinearity

EXAM TRAP: Multiple regression uses **matrix algebra**, not separate regressions!

TOPIC: Interpreting OLS Goodness of Fit

RSS INTERPRETATION:

- **Low RSS:** Small residuals, good fit
- **High RSS:** Large residuals, poor fit
- **$RSS = 0$:** Perfect fit (rare)

CAVEATS:

- Low training RSS could indicate overfitting
- Need to check test/validation performance
- Compare to baseline models

SALARY EXAMPLE:

- Large residuals observed
- Suggested OLS not ideal fit
- Motivated considering alternative models

RELATED METRICS:

- R-squared: Proportion of variance explained
- Adjusted R-squared: Penalizes additional parameters

EXAM TRAP: Low RSS = **good fit**, but check for overfitting!

TOPIC: Key Limitations of OLS

LINEARITY REQUIREMENT:

- OLS requires model linear in parameters
- $y = \beta_0 + \beta_1 x + \beta_2 x^2$ is OK (linear in β s)
- $y = \beta_0 + \beta_1 e^{\beta_2 x}$ is NOT OK (nonlinear in β_2)

OTHER LIMITATIONS:

- Sensitive to outliers
- Assumes homoscedasticity (constant error variance)
- Assumes no autocorrelation
- No closed-form for nonlinear models

SOLUTION FOR NONLINEAR:

- Use Nonlinear Least Squares (NLS)
- Use numerical optimization
- Use Maximum Likelihood Estimation

BINARY OUTCOMES:

- OLS cannot handle binary targets
- Use logistic regression (MLE)

EXAM TRAP: OLS requires **linearity in parameters**, not variables!

TOPIC: Comparing OLS and NLS

KEY DIFFERENCES:

Aspect	OLS	NLS
Linearity	Linear in parameters	Nonlinear in parameters
Solution	Closed-form	Iterative
Method	Analytical	Numerical
Speed	Fast	Slower
Example	$y = \beta_0 + \beta_1 x$	$y = \beta_0 + \beta_1 e^{\beta_2 x}$

COMMON GROUND:

- Both minimize sum of squared errors
- Both are least squares methods

WHEN TO USE EACH:

- OLS:** Simple linear regression, polynomial regression
- NLS:** Neural networks, exponential models

EXAM TRAP: OLS = closed-form; NLS = iterative!

TOPIC: Nonlinear Least Squares - Definition and Process

DEFINITION:

- Iterative numerical approach to estimate parameters in nonlinear models
- Minimizes objective function (RSS or MSE)
- No closed-form solution exists

NLS PROCESS:

- 1 Start with initial "guesses" for parameters (small random numbers)
- 2 Make predictions and evaluate objective function
- 3 Adjust parameters to improve objective
- 4 Check convergence; repeat if needed

WHY ITERATIVE:

- No closed-form solution
- Normal equations are nonlinear
- Requires numerical optimization

APPLICATIONS:

- Neural networks
- Complex curve fitting
- Models with exponential terms

EXAM TRAP: NLS is **iterative** because no closed-form solution exists!

TOPIC: Hill Climbing in NLS Optimization

DEFINITION:

- Simple method for estimating parameters
- Tests small changes to each parameter one at a time
- No derivatives required

APPROACH:

- 1 Start with initial guesses
- 2 Test small changes to each parameter
- 3 Keep changes that improve objective
- 4 Repeat until no improvement

ADVANTAGES:

- Simple to implement
- No derivatives needed
- Works for non-differentiable functions

DISADVANTAGES:

- Gets stuck in local optima
- Slow convergence
- Only one parameter at a time
- Misses optimal combinations

EXAM TRAP: Hill climbing is **simple but prone to local optima!**

TOPIC: Why Gradient Descent is Preferred Over Hill Climbing

GRADIENT DESCENT ADVANTAGES:

- Updates all parameters simultaneously
- Uses gradient information (steepest descent)
- Faster convergence than hill climbing
- More efficient for complex models

HILL CLIMBING LIMITATIONS:

- Only one parameter at a time
- Misses optimal parameter combinations
- Slower convergence

COMMON GROUND:

- Both can get stuck in local minima
- Neither guarantees global minimum

WHY GRADIENT DESCENT WINS:

- Workhorse of modern machine learning
- Scales to high dimensions
- Foundation of neural network training

EXAM TRAP: Gradient descent updates **all parameters simultaneously!**

TOPIC: Importance of Initial Guesses in NLS

INITIAL GUESSES:

- Starting values for parameters
- Often small random numbers
- Critical for convergence

WHY THEY MATTER:

- No closed-form solution
- Iterative process needs starting point
- Different starts may give different results

POTENTIAL ISSUES:

- Bad starting values → slow convergence
- May converge to local minimum
- May not converge at all

BEST PRACTICES:

- Try multiple random starts
- Use domain knowledge
- Normalize data first

EXAM TRAP: NLS starts with **initial guesses**, not exact values!

TOPIC: Convergence Criteria in NLS

DEFINITION:

- Conditions for stopping the iterative process
- Determines when solution is "good enough"

COMMON CRITERIA:

- 1 **Objective improvement:** Change in loss below threshold
- 2 **Gradient magnitude:** Gradient near zero
- 3 **Parameter change:** Parameters stop changing
- 4 **Maximum iterations:** Prevents infinite loops

TRADE-OFFS:

- Tight threshold: More iterations, better precision
- Loose threshold: Fewer iterations, less precision

EXAMPLE:

- Stop when gradient ≤ 0.001
- Stop after 1000 iterations
- Stop when loss improves ≤ 0.0001

EXAM TRAP: Multiple stopping criteria exist, not just one!

TOPIC: Maximum Likelihood Estimation - Definition and Overview

DEFINITION:

- Probabilistic approach to parameter estimation
- Finds parameter values that maximize probability of observing data
- More flexible than Least Squares

CORE QUESTION:

- "Given observed data, what parameter values make this data most probable?"

PROCESS:

- 1 Assume data from specific distribution
- 2 Construct likelihood function: $L(\theta) = P(X|\theta)$
- 3 Find θ that maximizes $L(\theta)$
- 4 Use log-likelihood for easier computation

KEY PROPERTIES:

- Consistent
- Asymptotically efficient
- Asymptotically normal

EXAM TRAP: MLE maximizes **likelihood** = $P(\text{data}|\theta)$!

TOPIC: When OLS and MLE Give Identical Estimates

EQUIVALENCE CONDITION:

- Errors are independently normally distributed
- Zero mean
- Constant variance

WHY EQUIVALENCE:

- Log-likelihood: $\ell(\beta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (y_i - X_i\beta)^2$
- Maximizing $\ell(\beta) \equiv$ minimizing $\sum (y_i - X_i\beta)^2$
- Same objective function

WHEN THEY DIFFER:

- Non-normal errors
- Heteroscedasticity
- Different distributions (Bernoulli, Poisson, etc.)

PRACTICAL IMPLICATION:

- Can use either method for linear regression
- MLE more flexible for other models

EXAM TRAP: OLS = MLE under **normal errors**!

TOPIC: Understanding the Likelihood Function

DEFINITION:

- $L(\theta|X) = P(X|\theta)$
- Probability of observing data given parameters
- Product of individual probabilities (if independent)

FORMULA:

$$L(\theta) = \prod_{i=1}^N f(x_i|\theta)$$

where $f(x_i|\theta)$ is the probability density/mass function

KEY DISTINCTION:

- **Likelihood:** $P(\text{data}|\theta)$
- **Posterior:** $P(\theta|\text{data})$ (Bayesian)

INDEPENDENCE ASSUMPTION:

- Joint probability = product of individual probabilities
- $\prod$ notation for multiplication

EXAM TRAP: Likelihood = $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$!

TOPIC: Why Use Log-Likelihood Instead of Likelihood

DEFINITION:

- Natural logarithm of the likelihood function
- $\ell(\theta) = \log L(\theta) = \sum_{i=1}^N \log f(x_i|\theta)$

MATHEMATICAL CONVENIENCE:

- Likelihood: $L(\theta) = \prod f(x_i|\theta)$
- Log-likelihood: $\ell(\theta) = \sum \log f(x_i|\theta)$
- Products become sums

MONOTONICITY:

- $\log$ is strictly increasing
- Maximizing $\ell(\theta) \equiv$ maximizing $L(\theta)$

NUMERICAL STABILITY:

- Products of small probabilities underflow
- Sums of logs are stable

EXAM TRAP: Log transforms **products to sums!**

TOPIC: MLE for Logistic Regression

BINARY OUTCOME:

- $y_i \in \{0, 1\}$
- Bernoulli distribution

BERNOULLI LIKELIHOOD:

$$L(\beta) = \prod_{i=1}^N p_i^{y_i} (1 - p_i)^{1-y_i}$$

where $p_i = \frac{1}{1+e^{-x_i\beta}}$

LOG-LIKELIHOOD:

$$\ell(\beta) = \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

MAXIMIZATION:

- No closed-form solution
- Numerical optimization (Newton-Raphson, IRLS)

EXAM TRAP: Logistic regression uses **Bernoulli** distribution!

TOPIC: MLE for Linear Regression

CONTINUOUS OUTCOME:

- $y_i \in \mathbb{R}$ (real numbers)
- Normal distribution for errors

NORMAL LIKELIHOOD:

$$L(\beta, \sigma^2) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i\beta)^2}{2\sigma^2}\right)$$

LOG-LIKELIHOOD:

$$\ell(\beta) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (y_i - X_i\beta)^2$$

EQUIVALENCE TO OLS:

- Maximizing ℓ = minimizing RSS
- Same parameter estimates

EXAM TRAP: Linear regression uses **Normal** distribution!

TOPIC: Log-Likelihood Function for Logit Model

LOGIT MODEL:

- $P_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{1i} + \dots + \beta_m x_{mi})}}$
- $y_i = 1$ with probability P_i
- $y_i = 0$ with probability $1 - P_i$

LIKELIHOOD FOR ONE OBSERVATION:

$$L(y_i) = F(y_i)^{y_i} (1 - F(y_i))^{1-y_i}$$

JOINT LIKELIHOOD:

$$L = \prod_{i=1}^N F(y_i)^{y_i} (1 - F(y_i))^{1-y_i}$$

LOG-LIKELIHOOD:

$$\log L = \sum_{i=1}^N [y_i \log(F(y_i)) + (1 - y_i) \log(1 - F(y_i))]$$

EXAM TRAP: Logit log-likelihood uses **log of F** and **1-F**!

TOPIC: Threshold Z in Classification

DEFINITION:

- Threshold Z converts predicted probabilities into class predictions
- Classification rule: $\hat{y}_i = 1$ if $P_i \geq Z$, else $\hat{y}_i = 0$

COMMON THRESHOLDS:

- $Z = 0.5$: Balanced costs
- Lower Z : More conservative (predict more positives)
- Higher Z : More liberal (predict fewer positives)

EXAMPLE (LOAN DEFAULT):

- $Z = 0.05$ (5%)
- Because cost of bad loan $\gg$ profit foregone
- More conservative lending decisions

KEY INSIGHT:

- Threshold depends on relative costs
- Lower threshold when false negatives more costly

EXAM TRAP: Threshold converts probabilities to class labels!

TOPIC: Cost Asymmetry and Threshold Selection

TYPE I ERROR (FALSE POSITIVE):

- Predict default, actually no default
- Cost = Lost profit from not making loan

TYPE II ERROR (FALSE NEGATIVE):

- Predict no default, actually defaults
- Cost = Actual loan loss (much larger)

RATIONALE FOR LOW THRESHOLD:

- More conservative (predict more defaults)
- Catches more actual defaults
- Accepts more false positives

GENERAL PRINCIPLE:

- Threshold depends on relative costs
- Lower threshold when FN more costly

EXAM TRAP: Lower threshold when missing positive is costly!

TOPIC: Key Advantages of MLE Over OLS

MLE FLEXIBILITY:

- Works with any distribution
- Bernoulli (logistic regression)
- Normal (linear regression)
- Poisson (count data)
- GARCH (volatility)

OLS LIMITATIONS:

- Only for continuous outcomes
- Assumes normal errors
- Cannot handle binary outcomes

WHEN TO USE EACH:

- **OLS**: Simple linear regression
- **MLE**: Complex probabilistic models

EXAM TRAP: MLE is **more flexible** than OLS!

TOPIC: Why OLS Cannot Handle Binary Outcomes

BINARY OUTCOME ISSUES:

- $y_i \in \{0, 1\}$ (not continuous)
- Errors not normally distributed
- Predictions can be outside $[0, 1]$

PROBLEMS WITH OLS:

- Heteroscedastic errors
- Inefficient estimates
- Invalid predictions

SOLUTION:

- Use logistic regression (MLE)
- Predict probabilities bounded $[0, 1]$

EXAMPLE:

- Loan default: 0 = no default, 1 = default
- OLS would predict values like -0.3 or 1.5
- Logistic regression predicts probabilities

EXAM TRAP: OLS inappropriate for **binary outcomes**!

TOPIC: Log-Likelihood Formula for Logistic Regression

FORMULA:

$$\ell = \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \frac{1}{1 + e^{-x_i \beta}}$

INTERPRETATION:

- For $y_i = 1$: $\log(p_i)$
- For $y_i = 0$: $\log(1 - p_i)$

MAXIMIZATION:

- Numerical optimization needed
- No closed-form solution
- Newton-Raphson or IRLS

PROPERTIES:

- Concave function
- Unique maximum
- Fast convergence

EXAM TRAP: Know the [log-likelihood formula](#) for logistic regression!

TOPIC: Optimization Methods for MLE in Logistic Regression

NO CLOSED-FORM:

- Log-likelihood is nonlinear in parameters
- Cannot solve analytically

NUMERICAL METHODS:

- **Newton-Raphson**: Uses second derivatives (Hessian)
- **IRLS**: Iteratively Reweighted Least Squares
- **Gradient Descent**: First-order method

CONVERGENCE:

- Usually converges quickly
- May fail with perfect separation

PERFECT SEPARATION:

- When classes perfectly separated
- Coefficients diverge to infinity
- Need regularization

EXAM TRAP: Logistic regression requires **numerical optimization**!

TOPIC: Consistency Property of MLE

DEFINITION:

- Estimator converges to true parameter as sample size increases
- $\hat{\theta}_{MLE} \xrightarrow{P} \theta_0$ as $n \rightarrow \infty$

MLE PROPERTIES:

- **Consistent:** Converges to true value
- **Asymptotically efficient:** Minimum variance
- **Asymptotically normal:** Normal distribution

CAVEAT:

- MLE can be biased in finite samples
- Example: $\hat{\sigma}_{MLE}^2$ is biased for σ^2
- Consistency is asymptotic property

WHY IT MATTERS:

- Reliable estimates with large samples
- Foundation for inference

EXAM TRAP: MLE is **consistent** but NOT necessarily unbiased!

TOPIC: Asymptotic Distribution of MLE

FORMULA:

$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \xrightarrow{d} N(0, I^{-1}(\theta_0))$$

where $I(\theta)$ = Fisher Information matrix

IMPLICATIONS:

- Confidence intervals can be constructed
- Hypothesis tests can be performed
- MLE achieves efficiency bound

FISHER INFORMATION:

$$I(\theta) = -E \left[\frac{\partial^2 \ell(\theta)}{\partial \theta^2} \right]$$

PRACTICAL USE:

- Standard errors from inverse Hessian
- Wald tests, likelihood ratio tests

EXAM TRAP: MLE is **asymptotically normal**!

TOPIC: Asymptotic Efficiency of MLE

DEFINITION:

- MLE achieves lowest possible variance among consistent estimators asymptotically
- No consistent estimator has lower variance

CRAMÉR-RAO LOWER BOUND:

- Minimum variance for unbiased estimators
- MLE achieves this asymptotically
- $Var(\hat{\theta}) \geq \frac{1}{I(\theta)}$

CAVEATS:

- Finite sample properties may differ
- Regularity conditions must hold

WHY IT MATTERS:

- Most precise estimates possible
- Efficient use of data

EXAM TRAP: MLE is **asymptotically efficient**!

TOPIC: Invariance Property of MLE

DEFINITION:

- If $\hat{\theta}_{MLE}$ is MLE of θ
- Then $g(\hat{\theta}_{MLE})$ is MLE of $g(\theta)$
- For any continuous function g

EXAMPLE:

- If $\hat{\sigma}^2$ is MLE of σ^2
- Then $\sqrt{\hat{\sigma}^2}$ is MLE of σ

WHY USEFUL:

- Easy transformation of estimates
- No need to re-estimate for transformed parameters

IMPORTANT:

- Invariance does NOT hold for expected values
- $E[g(\hat{\theta})] \neq g(E[\hat{\theta}])$ generally

EXAM TRAP: MLE is **invariant** to parameter transformations!

TOPIC: Fisher Information in MLE

DEFINITION:

- Measure of amount of information data carries about unknown parameter
- Higher information = more precise estimates

FORMULA:

$$I(\theta) = -E \left[\frac{\partial^2 \ell(\theta)}{\partial \theta^2} \right]$$

CONNECTION TO VARIANCE:

- $\text{Var}(\hat{\theta}) \geq \frac{1}{I(\theta)}$
- Cramér-Rao lower bound

INTERPRETATION:

- More data = more information
- Sharper likelihood peak = more information

EXAM TRAP: Fisher Information measures **information about parameters!**

TOPIC: Applications of MLE

LINEAR REGRESSION:

- Can use OLS or MLE
- Equivalent under normal errors

LOGISTIC REGRESSION:

- Uses MLE (Bernoulli likelihood)
- No OLS equivalent

OTHER MLE APPLICATIONS:

- Probit models
- GARCH models
- Count data models
- Survival analysis

KEY INSIGHT:

- MLE is flexible framework
- Works with any distribution
- Foundation for many models

EXAM TRAP: Both linear and logistic regression can use **MLE**!

TOPIC: Key Advantages of MLE Over OLS

FLEXIBILITY OF MLE:

- Works with any distribution
- Handles binary, count, continuous outcomes
- Used in complex models (GARCH, neural networks)

LIMITATIONS OF OLS:

- Only for continuous outcomes
- Assumes normal errors
- Cannot handle binary outcomes

PRACTICAL GUIDANCE:

- **OLS:** Simple linear regression
- **MLE:** Complex probabilistic models

REMEMBER:

- MLE more flexible but more complex
- OLS simpler for basic regression

EXAM TRAP: MLE is **more flexible** than OLS!

T41: Gradient Descent Definition

TOPIC: Gradient Descent - Definition and Overview

DEFINITION:

- Iterative optimization algorithm
- Updates parameters in direction of negative gradient
- Workhorse of modern machine learning

CORE CONCEPT:

- Move in direction of steepest descent
- Update rule: $w^{new} = w^{old} - \eta \frac{\partial L}{\partial w}$
- η = learning rate
- $\frac{\partial L}{\partial w}$ = gradient

PROCESS:

- 1 Start with initial guesses
- 2 Calculate gradient
- 3 Update parameters in opposite direction
- 4 Repeat until convergence

CONVERGENCE:

- Gradient near zero
- Loss no longer decreasing

EXAM TRAP: Gradient descent uses **negative gradient** direction!

TOPIC: Gradient Descent Update Rule

FORMULA:

$$w_i^{new} = w_i^{old} - \eta \frac{\partial \mathcal{L}(w_i)}{\partial w_i}$$

COMPONENTS:

- w_i : Parameter to update
- η : Learning rate (0, 1)
- $\frac{\partial \mathcal{L}}{\partial w_i}$: Partial derivative (gradient)

WHY SUBTRACT:

- Gradient points uphill
- Negative gradient points downhill
- Subtracting moves toward minimum

EXAMPLE:

- Current $w = 0.8$, gradient = -48.66, $\eta = 0.005$
- Update: $0.8 - 0.005(-48.66) = 1.043$

EXAM TRAP: Gradient descent **subtracts** gradient!

TOPIC: Learning Rate - Too Small

SMALL LEARNING RATE:

- Small steps
- Slow convergence
- Many iterations needed
- May get stuck in local minima

VISUAL:

- Tiny steps down the valley
- Takes forever to reach bottom

SOLUTION:

- Increase learning rate
- Use dynamic learning rate

EXAMPLE:

- $\eta = 0.0001$: Very slow
- $\eta = 0.001$: Still slow
- $\eta = 0.01$: Reasonable

EXAM TRAP: Too small η = slow convergence!

T44: Gradient Descent - Too Large Learning Rate

TOPIC: Learning Rate - Too Large

LARGE LEARNING RATE:

- Large steps
- Overshooting minimum
- Oscillation around minimum
- Divergence (loss increases)

VISUAL:

- Jumping over the valley
- Bouncing back and forth

SOLUTION:

- Decrease learning rate
- Use dynamic learning rate

EXAMPLE:

- $\eta = 10$: Diverges
- $\eta = 1$: Oscillates
- $\eta = 0.1$: Maybe OK

EXAM TRAP: Too large η = overshooting/divergence!

TOPIC: Epoch in Gradient Descent

DEFINITION:

- One complete pass through the entire training dataset
- Includes calculating loss and adjusting weights

BATCH GRADIENT DESCENT:

- One epoch = one weight update
- Uses all data for each update

STOCHASTIC GRADIENT DESCENT:

- One epoch = N weight updates
- Uses one data point per update

EXAMPLE:

- Training data: 1000 observations
- Batch GD: 1 update per epoch
- SGD: 1000 updates per epoch

EXAM TRAP: Epoch = **one complete pass** through data!

TOPIC: Batch Gradient Descent

DEFINITION:

- Uses entire training dataset for each update
- Computes gradient over all N samples

CHARACTERISTICS:

- One update per epoch
- Stable convergence
- Deterministic

ADVANTAGES:

- Stable convergence
- Deterministic

DISADVANTAGES:

- Computationally expensive
- Requires all data in memory

EXAM TRAP: Batch GD uses **entire dataset** per update!

TOPIC: Stochastic Gradient Descent (SGD)

DEFINITION:

- Uses one randomly selected data point per update
- Updates weights after each observation

CHARACTERISTICS:

- N updates per epoch
- Noisy convergence
- Random selection

ADVANTAGES:

- Lower memory requirements
- Faster per iteration
- Can escape local minima (noise)

DISADVANTAGES:

- Noisy convergence
- More iterations needed

EXAM TRAP: SGD uses **one data point** per update!

TOPIC: Mini-Batch Gradient Descent

DEFINITION:

- Uses small subset (batch) of data for each update
- Balances batch and stochastic methods

CHARACTERISTICS:

- Batch size: 32, 64, 128
- Multiple updates per epoch
- Most common in practice

ADVANTAGES:

- More stable than SGD
- Faster than batch
- Efficient memory usage

COMMON IN PRACTICE:

- Standard for deep learning
- Used in most frameworks

EXAM TRAP: Mini-batch uses **small subset** of data!

TOPIC: Convergence in Gradient Descent

DEFINITION:

- Algorithm has reached a point where further updates don't improve loss significantly
- Gradient is near zero

CONVERGENCE CRITERIA:

- 1 Gradient magnitude below threshold
- 2 Loss improvement below threshold
- 3 Maximum iterations reached
- 4 Validation error starts increasing

PRACTICAL CONSIDERATIONS:

- Exact zero gradient rare
- Tradeoff: precision vs computation
- Early stopping prevents overfitting

EXAMPLE:

- Stop when $\text{gradient} \leq 0.001$
- Stop after 1000 iterations

EXAM TRAP: Convergence = **improvement below threshold**, not exactly zero gradient!

TOPIC: Local Minima in Gradient Descent

DEFINITION:

- Point where loss is lower than neighbors but not lowest possible
- Gradient = 0
- Not global minimum

PROBLEM:

- Gradient descent can get stuck
- May not find global minimum

SOLUTIONS:

- Momentum
- Multiple restarts
- Stochastic gradient descent
- Adaptive learning rates

IN HIGH DIMENSIONS:

- Saddle points more common than local minima
- Neural networks: local minima rare

EXAM TRAP: Local minima are **traps** for gradient descent!

TOPIC: Momentum in Gradient Descent

DEFINITION:

- Accelerates convergence by accumulating past gradients
- Reduces oscillations

UPDATE RULE:

$$w^{new} = w^{old} - \eta \frac{\partial \mathcal{L}}{\partial w^{old}} + \mu(w^{old} - w^{older})$$

where μ = momentum rate (0, 1)

BENEFITS:

- Speeds up convergence
- Reduces oscillation
- Helps escape local minima

INTUITION:

- Like a ball rolling down hill
- Builds up speed in consistent directions
- Dampens oscillations

EXAM TRAP: Momentum uses **past gradients** to smooth updates!

TOPIC: Dynamic Learning Rate

DEFINITION:

- Learning rate changes during training
- Start with larger η , reduce over time

DECAY FUNCTIONS:

- Exponential: $\eta_t = \eta_0 e^{-kt}$
- Inverse: $\eta_t = \frac{\eta_0}{1+kt}$

BENEFITS:

- Fast initial progress
- Fine-tuning near optimum
- Better convergence

EXAMPLE:

- Start: $\eta_0 = 0.1$
- After 100 epochs: $\eta = 0.01$
- After 1000 epochs: $\eta = 0.001$

EXAM TRAP: Dynamic learning rate **decays** over time!

TOPIC: Exponential Decay for Learning Rate

FORMULA:

$$\eta_t = \eta_0 e^{-kt}$$

COMPONENTS:

- η_0 : Initial learning rate
- k : Decay rate
- t : Epoch number

PROPERTIES:

- Smooth continuous decay
- Never reaches exactly zero
- Common in practice

EXAMPLE:

- $\eta_0 = 0.1, k = 0.01$
- $t = 0: \eta = 0.1$
- $t = 100: \eta = 0.1e^{-1} = 0.037$
- $t = 200: \eta = 0.1e^{-2} = 0.014$

EXAM TRAP: Know the [exponential decay formula](#)!

TOPIC: Inverse Decay for Learning Rate

FORMULA:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

COMPONENTS:

- η_0 : Initial learning rate
- k : Decay rate
- t : Epoch number

PROPERTIES:

- Slower decay than exponential
- Approaches zero asymptotically

EXAMPLE:

- $\eta_0 = 0.1, k = 0.01$
- $t = 0: \eta = 0.1$
- $t = 100: \eta = 0.1/2 = 0.05$
- $t = 200: \eta = 0.1/3 = 0.033$

EXAM TRAP: Inverse decay: $\eta_t = \frac{\eta_0}{1+kt}$!

TOPIC: Gradient Descent Example - Salary Data

EXAMPLE SETUP:

- Known: $b_0 = 16.88$
- Optimize: b_1
- Initial: $b_1 = 0.8$
- Learning rate: $\eta = 0.005$

OBJECTIVE FUNCTION:

$$MSE(b_1) = \frac{1}{N} \sum_{i=1}^N (y_i - 16.88 - b_1 x_i)^2$$

GRADIENT:

$$\frac{\partial MSE}{\partial b_1} = -\frac{2}{N} \sum_{i=1}^N (y_i - \hat{y}_i) x_i$$

ITERATION 0:

- $b_1 = 0.8$
- Gradient = -48.664
- Update: $0.8 - 0.005(-48.664) = 1.043$

EXAM TRAP: Initial $b_1 = 0.8$, final $b_1 = 1.059$!

TOPIC: Convergence in Salary Example

CONVERGENCE TABLE:

Iteration	b_1	Gradient
0	0.800	-48.664
1	1.043	-3.024
2	1.058	-0.188
3	1.059	-0.012
4	1.059	-0.001
5	1.059	0.000

CONVERGENCE:

- Gradient approaches zero
- b_1 stabilizes at 1.059
- Only 5 iterations needed

COMPARISON:

- OLS solution: $\hat{b}_1 = 1.06$
- Gradient descent: 1.059 (approximation)

EXAM TRAP: Converged in 5 iterations in the example!

TOPIC: Epoch in Machine Learning

DEFINITION:

- One iteration including calculating loss and adjusting weights using whole training sample

FOR BATCH GRADIENT DESCENT:

- 1 epoch = 1 weight update

FOR STOCHASTIC GRADIENT DESCENT:

- 1 epoch = N weight updates

EXAMPLE:

- Training data: 1000 observations
- Batch GD: 1 update per epoch
- SGD: 1000 updates per epoch

EXAM TRAP: Epoch = **one full pass** through training data!

TOPIC: Local vs Global Minimum

LOCAL MINIMUM:

- Gradient = 0
- Lower than immediate neighbors
- May not be lowest overall

GLOBAL MINIMUM:

- Lowest point of entire function
- What we ideally want

CHALLENGE:

- Gradient descent can get stuck in local minima
- No guarantee of finding global minimum

SOLUTIONS:

- Momentum
- Multiple random starts
- Stochastic gradient descent

EXAM TRAP: Gradient descent may find **local**, **not global**, minimum!

TOPIC: Plateaus in Gradient Descent

DEFINITION:

- Flat region where gradient is near zero
- Not a minimum

PROBLEM:

- Gradient descent slows down
- May appear converged but isn't

SOLUTION:

- Momentum helps
- Adaptive learning rates

EXAMPLE:

- Flat region in loss landscape
- Gradient small but not zero
- Updates tiny

EXAM TRAP: Plateaus **slow down** gradient descent!

TOPIC: Key Advantages of Gradient Descent

KEY ADVANTAGE:

- Can optimize complex, high-dimensional, nonlinear models
- Handles complex models
- Works with high dimensions
- No closed-form needed

APPLICATIONS:

- Neural networks
- Logistic regression
- Any differentiable model

LIMITATIONS:

- Can get stuck in local minima
- Sensitive to learning rate
- May converge slowly

EXAM TRAP: Gradient descent = **workhorse** of modern ML!

T61: Backpropagation Definition

TOPIC: Backpropagation - Definition and Overview

DEFINITION:

- Algorithm that computes gradients of loss function with respect to all weights
- Uses chain rule of differentiation
- Foundation of neural network training

PROCESS:

- 1 Forward pass: compute output and loss
- 2 Backward pass: compute gradients
- 3 Update weights using gradient descent

KEY INSIGHT:

- Consider each layer separately
- Apply chain rule at each step
- Propagate errors backward

WHY "BACK":

- Start at output layer
- Work backward through layers

EXAM TRAP: Backpropagation uses the [chain rule](#)!

TOPIC: Chain Rule Foundation of Backpropagation

CHAIN RULE:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

NEURAL NETWORK APPLICATION:

- Output is function of function
- Multiple layers compose functions
- Chain rule computes gradients through composition

EXAMPLE:

- $L = f(g(h(x)))$
- $\frac{\partial L}{\partial x} = f'(g) \cdot g'(h) \cdot h'(x)$

WHY IT MATTERS:

- Enables gradient computation for deep networks
- Breaks complex problem into simpler steps
- Foundation of automatic differentiation

EXAM TRAP: Chain rule is the **foundation** of backpropagation!

TOPIC: Forward Pass in Backpropagation

DEFINITION:

- Input data passed through network to compute output and loss

PROCESS:

- 1 Input features fed to network
- 2 Hidden layer activations computed
- 3 Output computed
- 4 Loss calculated

EXAMPLE (HIDDEN LAYER):

$$H_j = f \left(\sum_i w_{i,j} x_i \right)$$

EXAMPLE (OUTPUT LAYER):

$$H_k = f \left(\sum_j w_{j,k} H_j \right)$$

EXAM TRAP: Forward pass computes **output and loss**!

TOPIC: Backward Pass in Backpropagation

DEFINITION:

- Gradients of loss with respect to weights computed using chain rule

PROCESS:

- 1 Start at output layer
- 2 Compute error at output
- 3 Propagate error backward
- 4 Compute gradients for each weight

EXAMPLE (OUTPUT LAYER):

$$e_k = -(y - H_k) \cdot f'(z_k)$$

EXAMPLE (HIDDEN LAYER):

$$e_j = e_k \cdot w_{j,k} \cdot f'(z_j)$$

EXAM TRAP: Backward pass computes **gradients**!

TOPIC: Weight Update in Backpropagation

UPDATE RULE:

$$w_{j,k}^{new} = w_{j,k}^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_{j,k}}$$

USING ERRORS:

$$w_{j,k}^{new} = w_{j,k}^{old} - \eta \cdot e_k \cdot H_j$$

PROCESS:

- 1 Compute gradient
- 2 Multiply by learning rate
- 3 Subtract from current weight

EXAMPLE:

- $w = 0.01$, $\eta = 0.1$, $e_k = -15.44$, $H_j = 0.5$
- $w^{new} = 0.01 - 0.1(-15.44)(0.5) = 0.782$

EXAM TRAP: Weights updated by **subtracting** gradient $\times$ learning rate!

TOPIC: Sigmoid Activation Function Derivative

SIGMOID FUNCTION:

$$f(z) = \frac{1}{1 + e^{-z}}$$

DERIVATIVE:

$$f'(z) = f(z)(1 - f(z))$$

DERIVATION:

- $f(z) = (1 + e^{-z})^{-1}$
- $f'(z) = -1(1 + e^{-z})^{-2} \cdot (-e^{-z})$
- $= \frac{e^{-z}}{(1 + e^{-z})^2}$
- $= f(z)(1 - f(z))$

IMPORTANCE:

- Used in backpropagation
- Easy to compute from $f(z)$

EXAM TRAP: Sigmoid derivative = $f(z)(1 - f(z))$!

TOPIC: Output Layer Error Calculation

FORMULA:

$$e_k = -(y - H_k) \cdot f'(z_k)$$

COMPONENTS:

- y : Actual target
- H_k : Predicted output
- $f'(z_k)$: Activation derivative

FOR LINEAR ACTIVATION:

- $f'(z_k) = 1$
- $e_k = -(y - H_k) = H_k - y$

EXAMPLE:

- $y = 15.46$, $H_k = 0.02$
- $e_k = -(15.46 - 0.02) = -15.44$

EXAM TRAP: Output error includes **activation derivative**!

T68: Backpropagation - Hidden Layer Error

TOPIC: Hidden Layer Error Calculation

FORMULA:

$$e_j = e_k \cdot w_{j,k} \cdot f'(z_j)$$

COMPONENTS:

- e_k : Error from next layer
- $w_{j,k}$: Weight connecting to next layer
- $f'(z_j)$: Activation derivative

EXAMPLE:

- $e_k = -15.44$, $w = 0.01$
- $f'(z_j) = 0.5 \times 0.5 = 0.25$
- $e_j = -15.44 \times 0.01 \times 0.25 = -0.0386$

PROCESS:

- Start with output error
- Work backward through layers
- Multiply by weights and derivatives

EXAM TRAP: Hidden error uses **weights and activation derivative!**

TOPIC: Summary of Backpropagation Steps

CORRECT ORDER:

- 1 Forward pass: compute output
- 2 Error calculation: compare to actual
- 3 Backward pass: compute gradients
- 4 Update weights using gradient descent

REPEAT:

- For each data point
- Until convergence

CONVERGENCE CRITERIA:

- Loss below threshold
- Improvement below threshold
- Maximum iterations reached
- Validation error increases

EXAM TRAP: Forward → Error → Backward → Update!

TOPIC: When Does Backpropagation Stop?

CONVERGENCE CRITERIA:

- 1 Loss below threshold
- 2 Improvement below threshold
- 3 Maximum iterations reached
- 4 Validation error increases

EPOCHS:

- Multiple passes through data
- Number depends on problem

EXAMPLE:

- Stop when loss ≤ 0.001
- Stop after 100 epochs
- Stop when validation error increases

EXAM TRAP: Multiple stopping criteria, not just one!

T71: Backpropagation - Vanishing Gradient

TOPIC: Vanishing Gradient Problem

DEFINITION:

- Gradients become very small during backpropagation
- Especially in early layers
- Product of many small derivatives

CAUSE:

- Chain rule multiplies derivatives
- Sigmoid derivatives are < 1
- Product quickly approaches zero

SOLUTION:

- Use ReLU activation
- Batch normalization
- LSTM architectures

CONSEQUENCE:

- Early layers don't learn
- Training very slow

EXAM TRAP: Vanishing gradient = **very small gradients** in early layers!

TOPIC: Exploding Gradient Problem

DEFINITION:

- Gradients become very large during backpropagation
- Product of many large derivatives
- Unstable parameter updates

CONSEQUENCES:

- Loss diverges
- Training fails

SOLUTION:

- Gradient clipping
- Lower learning rate
- Batch normalization

OPPOSITE PROBLEM:

- Vanishing gradient (very small)

EXAM TRAP: Exploding gradient = very large gradients!

TOPIC: ReLU Activation Function

RELU FUNCTION:

$$f(z) = \max(0, z)$$

DERIVATIVE:

$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

ADVANTAGES:

- Derivative is 1 for positive values
- No vanishing gradient for positive values
- Computationally efficient

WHY IT HELPS:

- Avoids vanishing gradient problem
- Faster training
- Standard in modern networks

EXAM TRAP: ReLU avoids vanishing gradient!

TOPIC: Batch Normalization

DEFINITION:

- Adds normalization layers between hidden layers
- Normalizes layer inputs

PROCESS:

- Subtract mean
- Divide by standard deviation
- Apply between hidden layers

BENEFITS:

- Reduces internal covariate shift
- Helps with vanishing/exploding gradients
- Allows higher learning rates

ANALOGY:

- Like normalizing input data
- But applied to hidden layers

EXAM TRAP: Batch normalization helps with **gradient problems**!

TOPIC: Summary of Backpropagation

TRUE STATEMENTS:

- Computes gradients for all weights
- Uses chain rule
- Requires storing activations

FALSE STATEMENT:

- Backpropagation does NOT guarantee global minimum
- Can get stuck in local minima
- Can suffer from vanishing/exploding gradients

KEY POINTS:

- Foundation of neural network training
- Chain rule is essential
- Multiple computational challenges exist

EXAM TRAP: Backpropagation does **NOT** guarantee global minimum!

TOPIC: Bias-Variance Tradeoff - Definition

DEFINITION:

- Tradeoff between choosing a model that is not complex enough (underfitting) and one that is too complex (overfitting)
- Balances bias and variance

TOTAL ERROR:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

GOAL:

- Find optimal complexity
- Balance bias and variance

VISUAL:

- Simple model: high bias, low variance
- Complex model: low bias, high variance
- Optimal: balance

EXAM TRAP: Bias = underfitting; Variance = overfitting!

TOPIC: Bias in the Bias-Variance Tradeoff

DEFINITION:

- Error from model's assumptions and limitations
- High bias = underfitting
- Model too simple

EXAMPLE:

- Linear model for nonlinear data
- Cannot capture true relationship

CHARACTERISTICS:

- Poor training performance
- Poor test performance

WHY IT MATTERS:

- Model systematically wrong
- Predictions biased

EXAM TRAP: Bias = error from assumptions (underfitting)!

TOPIC: Variance in the Bias-Variance Tradeoff

DEFINITION:

- Error from sensitivity to fluctuations in training data
- High variance = overfitting
- Model too complex

EXAMPLE:

- High-degree polynomial
- Fits noise in training data

CHARACTERISTICS:

- Good training performance
- Poor test performance

WHY IT MATTERS:

- Model unstable
- Small data changes $\rightarrow$ big prediction changes

EXAM TRAP: Variance = error from sensitivity (overfitting)!

TOPIC: Underfitting - Definition and Causes

DEFINITION:

- Model too simple to capture underlying patterns
- High bias
- Fails to capture relevant patterns

EXAMPLE:

- Linear model for quadratic relationship
- Missing interaction terms

SYMPTOMS:

- Poor training performance
- Poor test performance

CAUSES:

- Too few features
- Model too simple
- Excessive regularization

EXAM TRAP: Underfitting = model too simple!

TOPIC: Overfitting - Definition and Causes

DEFINITION:

- Model too complex, fits noise in training data
- High variance
- Learns random noise

EXAMPLE:

- High-degree polynomial
- Neural network with too many parameters

SYMPTOMS:

- Good training performance
- Poor test performance

CAUSES:

- Too many parameters
- Too little data
- No regularization

EXAM TRAP: Overfitting = model too complex!

T81: Bias-Variance - Model Complexity

TOPIC: Effect of Model Complexity on Bias and Variance

AS COMPLEXITY INCREASES:

- Bias decreases (fits training data better)
- Variance increases (more sensitive to training data)

VISUAL:

- Simple model: high bias, low variance
- Complex model: low bias, high variance
- Optimal: balance

EXAMPLE:

- Linear regression: high bias, low variance
- 20th degree polynomial: low bias, high variance

KEY INSIGHT:

- Tradeoff is fundamental
- Cannot minimize both simultaneously

EXAM TRAP: More complexity = lower bias, higher variance!

TOPIC: Finding Optimal Model Complexity

DEFINITION:

- Complexity that minimizes total error
- $\text{Total error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$

TRADEOFF:

- Too simple: high bias
- Too complex: high variance
- Optimal: balance

HOW TO FIND:

- Cross-validation
- Validation set performance
- Regularization

EXAMPLE:

- Quadratic model often optimal
- Balances underfitting and overfitting

EXAM TRAP: Optimal = **minimum total error!**

TOPIC: Underfitting in House Price Example

LINEAR REGRESSION:

- Forces strictly decreasing relationship
- Cannot capture non-monotonic pattern
- Underfits the data

EVIDENCE:

- Training MSE: 166.67
- Test MSE: 161.12
- Both high

WHY:

- House prices non-monotonic in age
- Both very new and very old houses expensive
- Linear model can't capture U-shape

EXAM TRAP: Linear regression **underfits** house price data!

TOPIC: Overfitting in House Price Example

NINTH-DEGREE POLYNOMIAL:

- Captures data-specific noise
- Curved shape between 20-30 years
- Poor generalization

EVIDENCE:

- Training MSE: 122.40 (lowest)
- Test MSE: 139.39 (higher than quadratic)

LESSON:

- Complex models can overfit
- Test performance reveals overfitting

EXAM TRAP: Ninth-degree polynomial **overfits!**

TOPIC: Optimal Model in House Price Example

QUADRATIC REGRESSION:

- Captures U-shape pattern
- Training MSE: 131.81
- Test MSE: 136.57 (lowest)

WHY BEST:

- Balanced complexity
- Generalizes well
- Captures true relationship

COMPARISON:

- Linear: underfits (high bias)
- Quadratic: optimal balance
- Ninth-degree: overfits (high variance)

EXAM TRAP: Quadratic strikes **best balance**!

TOPIC: Regularization - Definition and Purpose

DEFINITION:

- Technique to reduce overfitting by adding penalty term to loss function
- Shrinks coefficients toward zero
- Prevents overfitting

LOSS FUNCTION:

$$L = \text{RSS} + \lambda \cdot \text{Penalty}$$

TYPES:

- Ridge (L2): $\lambda \sum \beta_j^2$
- LASSO (L1): $\lambda \sum |\beta_j|$
- Elastic Net: both

BENEFITS:

- Simplifies models
- Reduces overfitting
- Improves generalization

EXAM TRAP: Regularization adds a **penalty term**!

TOPIC: Ridge Regression (L2 Regularization)

LOSS FUNCTION:

$$L = \frac{1}{N} \sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m \hat{\beta}_j^2$$

CHARACTERISTICS:

- Shrinks coefficients
- Does NOT set to zero
- Handles multicollinearity

WHY L2:

- Squared penalty term
- Differentiable everywhere
- Analytic solution exists

EXAMPLE:

- OLS coefficient: -23.22
- Ridge ($\lambda = 0.1$): -6.55
- Ridge ($\lambda = 0.5$): -2.00

EXAM TRAP: Ridge **shrinks** but **doesn't zero** coefficients!

TOPIC: LASSO Regression (L1 Regularization)

LOSS FUNCTION:

$$L = \frac{1}{N} \sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |\hat{\beta}_j|$$

CHARACTERISTICS:

- Sets some coefficients to zero
- Performs feature selection
- Numerical optimization needed

WHY L1:

- Absolute value penalty
- Creates sparsity
- Not differentiable at zero

EXAMPLE:

- $\lambda = 0.01$: Some coefficients zero
- $\lambda = 0.1$: Only 1 non-zero

EXAM TRAP: LASSO sets coefficients to zero!

TOPIC: Elastic Net Regularization

DEFINITION:

- Combination of L1 and L2
- $\lambda_1 \sum \beta_j^2 + \lambda_2 \sum |\beta_j|$

LOSS FUNCTION:

$$L = \text{RSS} + \lambda_1 \sum \beta_j^2 + \lambda_2 \sum |\beta_j|$$

BENEFITS:

- Combines both penalties
- Shrinks and selects
- More flexible

MODERN PREFERENCE:

- Shift toward Elastic Net
- Away from pure LASSO/Ridge

EXAM TRAP: Elastic Net = L1 + L2!

TOPIC: Effect of Regularization Parameter λ

AS λ INCREASES:

- Penalty becomes stronger
- Coefficients shrink more
- Model becomes simpler

TRADEOFF:

- Small λ : little regularization
- Large λ : heavy regularization
- Optimal λ : balance

EXAMPLE:

- $\lambda = 0$: OLS
- $\lambda = 0.1$: Moderate shrinkage
- $\lambda = 10$: Heavy shrinkage

EXAM TRAP: Larger λ = simpler model!

TOPIC: LASSO as Feature Selection

LASSO PROPERTY:

- L1 penalty creates sparsity
- Some $\beta_j = 0$ exactly
- Those features removed

AS λ INCREASES:

- More coefficients become zero
- More features removed

EXAMPLE FROM TEXT:

- $\lambda = 0.1$: Only 1 non-zero coefficient

WHY USEFUL:

- Automatic feature selection
- Simpler models
- Easier interpretation

EXAM TRAP: LASSO removes features by zeroing coefficients!

TOPIC: Key Differences Between Ridge and LASSO

RIDGE (L2):

- Shrinks coefficients
- Does NOT set to zero
- Handles multicollinearity

LASSO (L1):

- Can set coefficients to zero
- Feature selection
- Numerical optimization

CHOICE:

- Ridge: reduce extreme estimates
- LASSO: remove unimportant features

COMPARISON TABLE:

	Ridge	LASSO
Penalty	L2	L1
Zeros	No	Yes
Feature	No	Yes

EXAM TRAP: Ridge = **shrink**; LASSO = **select**!

TOPIC: Regularization Example - Stock Returns

OLS RESULTS:

- Large coefficients
- Some opposite signs
- Indicates multicollinearity

EXAMPLE:

- USTB1M: -23.22
- USTB3M: +50.64
- Opposite signs for correlated features

SOLUTION:

- Ridge regression
- LASSO

EXAM TRAP: OLS gave large coefficients due to multicollinearity!

TOPIC: Ridge Regression Example

RIDGE RESULTS:

- Reduced coefficient magnitudes
- Higher λ = more shrinkage
- Some signs changed

EXAMPLE:

- OLS USTB1M: -23.22
- Ridge $\lambda = 0.1$: -6.55
- Ridge $\lambda = 0.5$: -2.00

WHY THIS HELPS:

- Reduces instability from multicollinearity
- More stable predictions

EXAM TRAP: Ridge **reduces coefficient magnitude!**

TOPIC: LASSO Regression Example

LASSO RESULTS:

- Set some coefficients to zero
- $\lambda = 0.1$: Only 1 non-zero
- Feature selection

EXAMPLE:

- USTB1M: 0
- USTB3M: 0
- USTB5Y: -0.71 (only non-zero)

BENEFIT:

- Simpler model
- Easier interpretation
- Automatic feature selection

EXAM TRAP: LASSO sets coefficients to zero!

TOPIC: K-Fold Cross-Validation

DEFINITION:

- Splitting data into k folds
- Training on $k-1$ folds, validating on 1 fold
- Repeating k times

K-FOLD CV:

- 1 Split data into k folds
- 2 For each fold:
 - Train on $k-1$ folds
 - Validate on 1 fold
- 3 Average results

COMMON VALUES:

- $k = 5$ or 10
- $k = N$ (LOOCV)

BENEFITS:

- Efficient use of data
- Reliable performance estimate

EXAM TRAP: K-fold = train on $k-1$, validate on 1 !

TOPIC: Stratified Cross-Validation

DEFINITION:

- Preserves class distribution in each fold
- Important for imbalanced data

WHY NEEDED:

- Random folds may miss rare class
- Distorted evaluation

EXAMPLE:

- 100 observations, 10 defaults
- 10-fold CV: 9 non-defaults, 1 default per fold

PROCESS:

- Sample positive and negative separately
- Preserve class ratio

EXAM TRAP: Stratified CV preserves **class distribution!**

TOPIC: Bootstrapping

DEFINITION:

- Sampling with replacement from original data
- Creates multiple bootstrap samples

PROCESS:

- 1 Create bootstrap sample
- 2 Train model
- 3 Validate on out-of-bag data
- 4 Repeat many times

KEY FACTS:

- Some observations repeated
- 37
- Large number of iterations

BENEFITS:

- Estimates variability
- Confidence intervals

EXAM TRAP: Bootstrapping samples **with replacement!**

TOPIC: Grid Search for Hyperparameter Tuning

DEFINITION:

- Exhaustive search over predefined hyperparameter grid
- Used with cross-validation

PROCESS:

- 1 Define grid of hyperparameter values
- 2 Evaluate all combinations
- 3 Use cross-validation
- 4 Select best

EXAMPLE:

- λ from 0 to 100, step 1
- 5-fold CV
- $101 \times 5 = 505$ model fits

DRAWBACK:

- Computationally expensive
- May overfit the grid

EXAM TRAP: Grid search is *exhaustive*!

TOPIC: Key Takeaways from Model Estimation

ESTIMATION METHODS:

- **OLS:** Linear models, closed-form
- **NLS:** Nonlinear models, iterative
- **MLE:** Probabilistic, flexible

OPTIMIZATION:

- **Gradient Descent:** Negative gradient, learning rate
- **Backpropagation:** Chain rule, neural networks
- **Momentum:** Accelerates convergence

MODEL SELECTION:

- **Bias-Variance:** Underfitting vs overfitting
- **Regularization:** Ridge (L2), LASSO (L1), Elastic Net
- **Cross-Validation:** K-fold, stratified, bootstrapping
- **Grid Search:** Hyperparameter tuning

Exam Success: Understand methods, tradeoffs, and when to use each!